Mini Compiler for MiniLang

Compiler Design Course Project Report

Implementation: Python + Flask | Difficulty: Intermediate

Abstract

An interactive compiler pipeline for MiniLang. Users edit source in a browser and inspect tokens, AST, semantic checks, intermediate representation, variable values, and output. Execution uses an AST interpreter, not native machine code.

Scope: this compiles MiniLang, not arbitrary Python, C, Java, or JavaScript.

1. Architecture

```
Browser editor -> Flask -> Lexer -> Parser -> AST -> Semantic analysis -> IR -> Interpreter -> Output
```

Frontend: templates/index.html and static/style.css. Flask route: app.py. Compiler phases: compiler.py. Tests: tests/test_compiler.py.

2. Compiler phases

Lexical analysis groups characters into tokens with source positions. Syntax analysis uses recursive descent to validate grammar and build an AST. Semantic analysis checks variable declaration/use. IR generation emits temporary assignments and labels. The interpreter evaluates the AST and returns output.

3. Language features

Declarations (`let`), assignments, print, if/else, while loops, numbers, strings, booleans, arithmetic, comparisons, and // comments.

4. Setup

```
python -m venv .venv\npip install -r requirements.txt\npython app.py\nOpen http://127.0.0.1:5000
```

5. Sample input/output

```
let x = 10;\nlet y = 3;\nprint(x * 2 + y);\n\nOutput: 23
```

6. Testing and limitations

Run python -m unittest discover -s tests -v. Tests cover arithmetic, conditionals, loops, undeclared variables, and missing semicolons. Limitations include no native code generation, functions, arrays, full type checking, or lexical block scope. Future work includes a bytecode VM, syntax highlighting, and richer type checking.

7. Conclusion

The editable frontend makes it possible to test multiple MiniLang programs and inspect each major compiler stage instead of relying on a fixed input.